BLOCKCHAIN ENGINEERING

ASSIGNMENT - 009

Name: Aman Sarkar

Ht.no: 2303A51273

Batch: 05

Objective -

To interact with a decentralized finance (DeFi) protocol on a blockchain testnet by writing and executing a smart contract that deposits and withdraws tokens, demonstrating basic DeFi operations without real funds.

Code -

assignment_9.py > ...

```python
import subprocess
import sys
import tkinter as tk
from tkinter import messagebox

# ---------- AUTO INSTALL ----------
def install(package):
    subprocess.check_call([sys.executable, "-m", "pip", "install", package])

for pkg in ["web3", "py-solc-x", "eth-tester", "py-evm"]:
    try:
        __import__(pkg.replace("-", "_"))
    except:
        install(pkg)

from web3 import Web3
from solcx import compile_source, install_solc
from eth_tester import EthereumTester
from web3.providers.eth_tester import EthereumTesterProvider
from eth_account import Account

# ---------- BLOCKCHAIN ----------
w3 = Web3(EthereumTesterProvider(EthereumTester()))
install_solc('0.8.0')
```

```python
# ---------- CONTRACT ----------
source = '''
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract MockDeFi {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint256 amount) public {
        require(balances[msg.sender] >= amount, "Insufficient balance");
        balances[msg.sender] -= amount;
        payable(msg.sender).transfer(amount);
    }
}
'''

compiled = compile_source(source, solc_version='0.8.0')
_, contract_interface = compiled.popitem()
```

```python
abi = contract_interface['abi']
bytecode = contract_interface['bin']

default_account = w3.eth.accounts[0]

Contract = w3.eth.contract(abi=abi, bytecode=bytecode)
tx_hash = Contract.constructor().transact({'from': default_account, 'gas': 3000000})
receipt = w3.eth.wait_for_transaction_receipt(tx_hash)
contract = w3.eth.contract(address=receipt.contractAddress, abi=abi)

# ---------- GUI ----------
root = tk.Tk()
root.title("DeFi Wallet")
root.geometry("500x550")
root.configure(bg="#0f172a")  # dark blue

# ---------- VARIABLES ----------
wallet_address = tk.StringVar()
private_key = tk.StringVar()
balance_var = tk.StringVar(value="Balance: 0 ETH")

# ---------- FUNCTIONS ----------
def create_wallet():
    acct = Account.create()
```

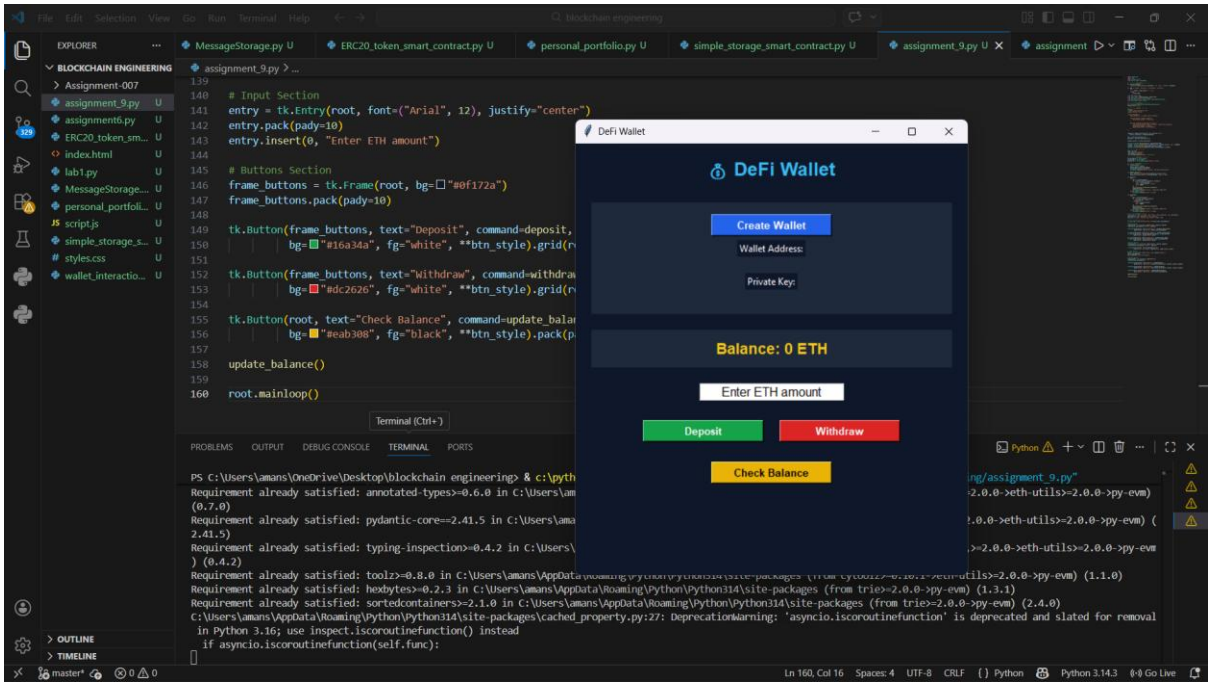
```python
    wallet_address.set(acct.address)
    private_key.set(acct.key.hex())
    messagebox.showinfo("Wallet Created", "New wallet generated!")

def update_balance():
    bal = contract.functions.balances(default_account).call()
    balance_var.set(f"Balance: {w3.from_wei(bal, 'ether')} ETH")

def deposit():
    try:
        amt = float(entry.get())
        tx = contract.functions.deposit().transact({
            'from': default_account,
            'value': w3.to_wei(amt, 'ether'),
            'gas': 200000
        })
        w3.eth.wait_for_transaction_receipt(tx)
        update_balance()
        messagebox.showinfo("Success", f"Deposited {amt} ETH")
    except Exception as e:
        messagebox.showerror("Error", str(e))

```

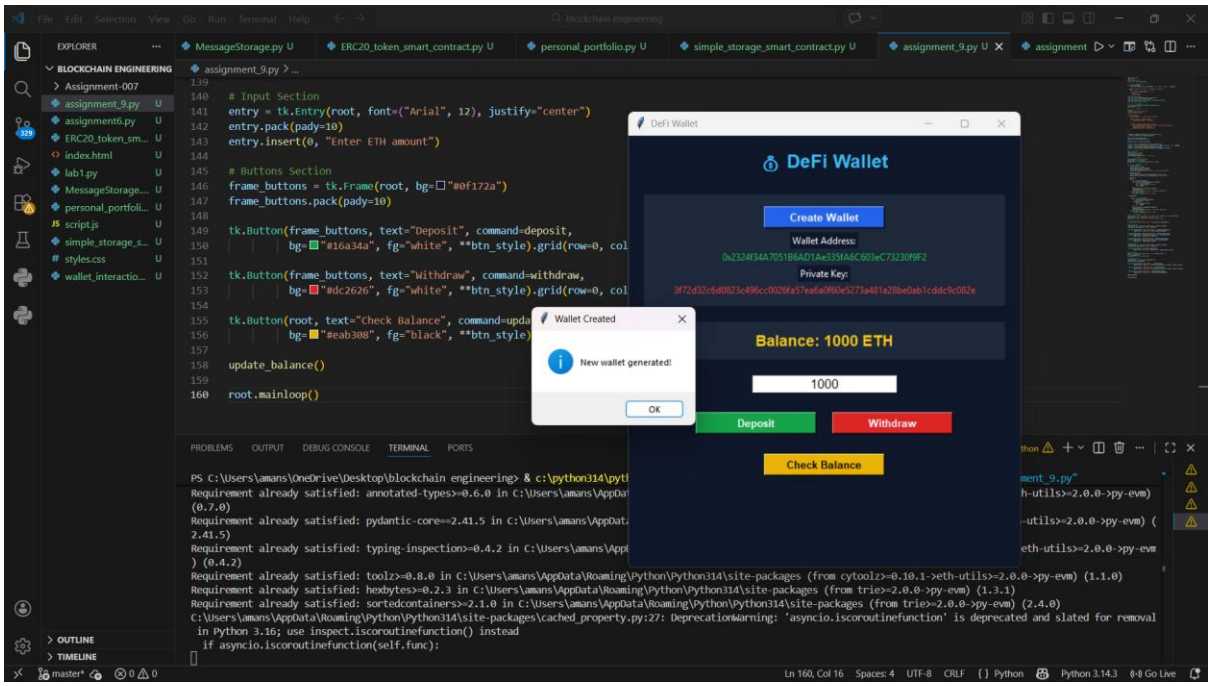
```python
def withdraw():
    try:
        amt = float(entry.get())
        tx = contract.functions.withdraw(
            w3.to_wei(amt, 'ether')
        ).transact({
            'from': default_account,
            'gas': 200000
        })
        w3.eth.wait_for_transaction_receipt(tx)
        update_balance()
        messagebox.showinfo("Success", f"Withdrawn {amt} ETH")
    except Exception as e:
        messagebox.showerror("Error", str(e))

# ---------- STYLES ----------
title_style = {"font": ("Arial", 18, "bold"), "bg": "#0f172a", "fg": "#38bdf8"}
label_style = {"bg": "#0f172a", "fg": "white"}
btn_style = {"font": ("Arial", 10, "bold"), "width": 18}

# ---------- UI ----------
tk.Label(root, text="💰 DeFi Wallet", **title_style).pack(pady=15)
```

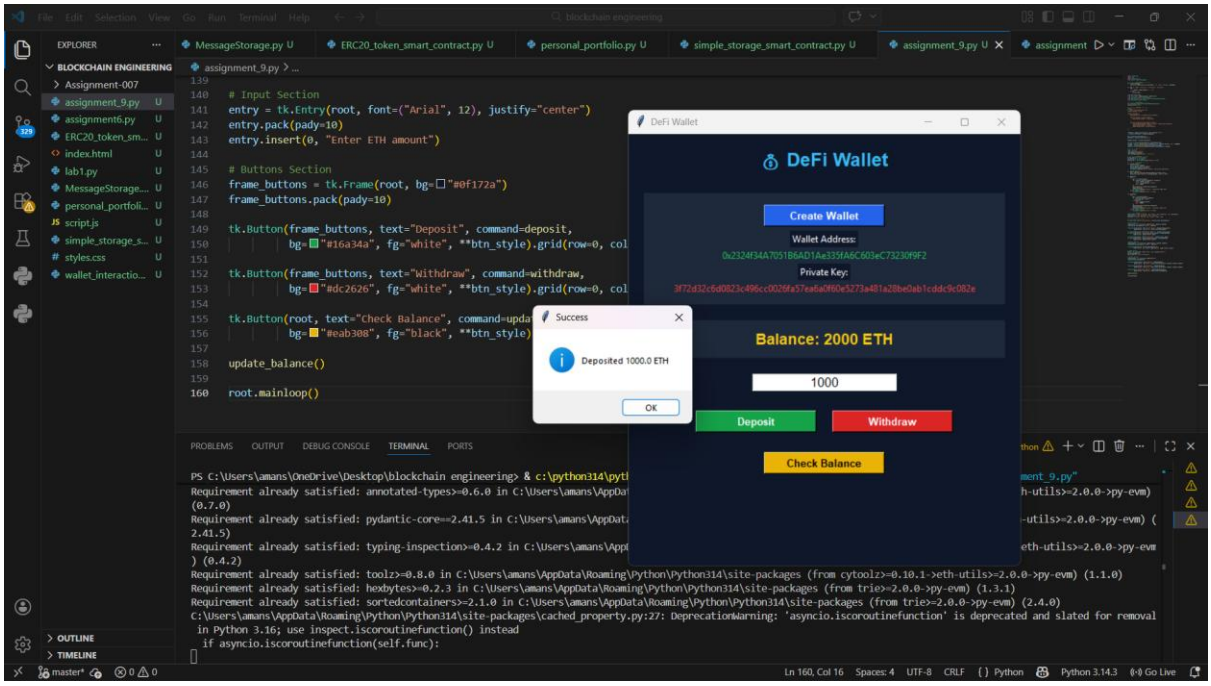
```python
# Wallet Section
frame_wallet = tk.Frame(root, bg="#1e293b", padx=10, pady=10)
frame_wallet.pack(pady=10, fill="x", padx=20)

tk.Button(frame_wallet, text="Create Wallet", command=create_wallet,
          bg="#2563eb", fg="white", **btn_style).pack(pady=5)

tk.Label(frame_wallet, text="Wallet Address:", **label_style).pack()
tk.Label(frame_wallet, textvariable=wallet_address, wraplength=400,
         fg="#22c55e", bg="#1e293b").pack()

tk.Label(frame_wallet, text="Private Key:", **label_style).pack()
tk.Label(frame_wallet, textvariable=private_key, wraplength=400,
         fg="#ef4444", bg="#1e293b").pack()

# Balance Section
frame_balance = tk.Frame(root, bg="#1e293b", padx=10, pady=10)
frame_balance.pack(pady=10, fill="x", padx=20)

tk.Label(frame_balance, textvariable=balance_var,
         font=("Arial", 14, "bold"), fg="#facc15", bg="#1e293b").pack()
```

```python
# Input Section
entry = tk.Entry(root, font=("Arial", 12), justify="center")
entry.pack(pady=10)
entry.insert(0, "Enter ETH amount")

# Buttons Section
frame_buttons = tk.Frame(root, bg="#0f172a")
frame_buttons.pack(pady=10)

tk.Button(frame_buttons, text="Deposit", command=deposit,
          bg="#16a34a", fg="white", **btn_style).grid(row=0, column=0, padx=10, pady=5)

tk.Button(frame_buttons, text="Withdraw", command=withdraw,
          bg="#dc2626", fg="white", **btn_style).grid(row=0, column=1, padx=10, pady=5)

tk.Button(root, text="Check Balance", command=update_balance,
          bg="#eab308", fg="black", **btn_style).pack(pady=10)

update_balance()

root.mainloop()
```

Output -


DeFi Wallet
Create Wallet
Wallet Address:
Private Key:
Balance: 0 ETH
Enter ETH amount
Deposit
Withdraw
Check Balance


DeFi Wallet
Create Wallet
Wallet Address:
0x2324f34A7051B6AD1Ae335fA6C603eC73230f9F2
Private Key:
3f72d32c6d0823c496cc0026fa57ea6a0f60e5273a481a28be0ab1cddc9c082e
Wallet Created
New wallet generated!
OK
Balance: 1000 ETH
1000
Deposit
Withdraw
Check Balance


assignment_9.py
# Input Section
entry = tk.Entry(root, font=("Arial", 12), justify="center")
entry.pack(pady=10)
entry.insert(0, "Enter ETH amount")
# Buttons Section
frame_buttons = tk.Frame(root, bg="#0f172a")
frame_buttons.pack(pady=10)
tk.Button(frame_buttons, text="Deposit", command=deposit,
bg="#16a34a", fg="white", **btn_style).grid(row=0, col
tk.Button(frame_buttons, text="Withdraw", command=withdraw,
bg="#dc2626", fg="white", **btn_style).grid(row=0, col
tk.Button(root, text="Check Balance", command=upda
bg="#eab308", fg="black", **btn_style)
update_balance()
root.mainloop()
DeFi Wallet
Create Wallet
Wallet Address:
0x2324f34A7051B6AD1Ae335fA6C603eC73230f9F2
Private Key:
3f72d32c6d0823c496cc0026fa57ea6a0f60e5273a481a28be0ab1cddc9c082e
Balance: 2000 ETH
1000
Deposit
Withdraw
Check Balance
Success
Deposited 1000.0 ETH
OK
PS C:\Users\amans\OneDrive\Desktop\blockchain engineering> & c:\python314\pyt
Requirement already satisfied: annotated-types>=0.6.0 in C:\Users\amans\AppDat
(0.7.0)
Requirement already satisfied: pydantic-core==2.41.5 in C:\Users\amans\AppDat
2.41.5)
Requirement already satisfied: typing-inspection>=0.4.2 in C:\Users\amans\App
) (0.4.2)
Requirement already satisfied: toolz>=0.8.0 in C:\Users\amans\AppData\Roaming\Python\Python314\site-packages (from cytoolz>=0.10.1->eth-utils>=2.0.0->py-evm) (1.1.0)
Requirement already satisfied: hexbytes>=0.2.3 in C:\Users\amans\AppData\Roaming\Python\Python314\site-packages (from trie>=2.0.0->py-evm) (1.3.1)
Requirement already satisfied: sortedcontainers>=2.1.0 in C:\Users\amans\AppData\Roaming\Python\Python314\site-packages (from trie>=2.0.0->py-evm) (2.4.0)
C:\Users\amans\AppData\Roaming\Python\Python314\site-packages\cached_property.py:27: DeprecationWarning: 'asyncio.iscoroutinefunction' is deprecated and slated for removal
in Python 3.16; use inspect.iscoroutinefunction() instead
if asyncio.iscoroutinefunction(self.func):